# Solution: Decimal to Binary Conversion

## Problem Statement

Given a positive integer `n`, write a function that returns its binary equivalent as a string. The function should not use any in-built binary conversion function.

## Examples

**Example 1**:

```
Input: 2
Output: "10"
Explanation: The binary equivalent of 2 is 10.
```

**Example 2**:

```
Input: 7
Output: "111"
Explanation: The binary equivalent of 7 is 111.
```

**Example 3**:

```
Input: 18
Output: "10010"
Explanation: The binary equivalent of 18 is 10010.
```

## Solution

We can use a stack to efficiently create the binary representation of a given decimal number. Our algorithm will take advantage of the 'Last In, First Out' property of stacks to reverse the order of the binary digits, since the binary representation is constructed from least significant bit to most significant bit, but needs to be displayed in the opposite order. The procedure involves repeatedly dividing the decimal number by 2, pushing the remainder onto the stack, which corresponds to the binary digit. When the number is reduced to 0, the algorithm pops elements off the stack and appends to the result string until the stack is empty, thereby reversing the order of digits. The result is the binary equivalent of the input decimal number.

Here is a detailed walkthrough of the solution:

1. First, the algorithm starts by creating an empty stack. A stack is chosen because of its "Last In, First Out" property which is perfect for this type of problem where we need to reverse the order of the operations.

2. Then, it enters into a loop where the given number is repeatedly divided by 2. This is because the binary number system is base 2, and each bit represents a power of 2.

3. Inside the loop, the remainder when the number is divided by 2 (which is either 0 or 1) is pushed onto the stack. This remainder is essentially the bit in the binary representation of the number.

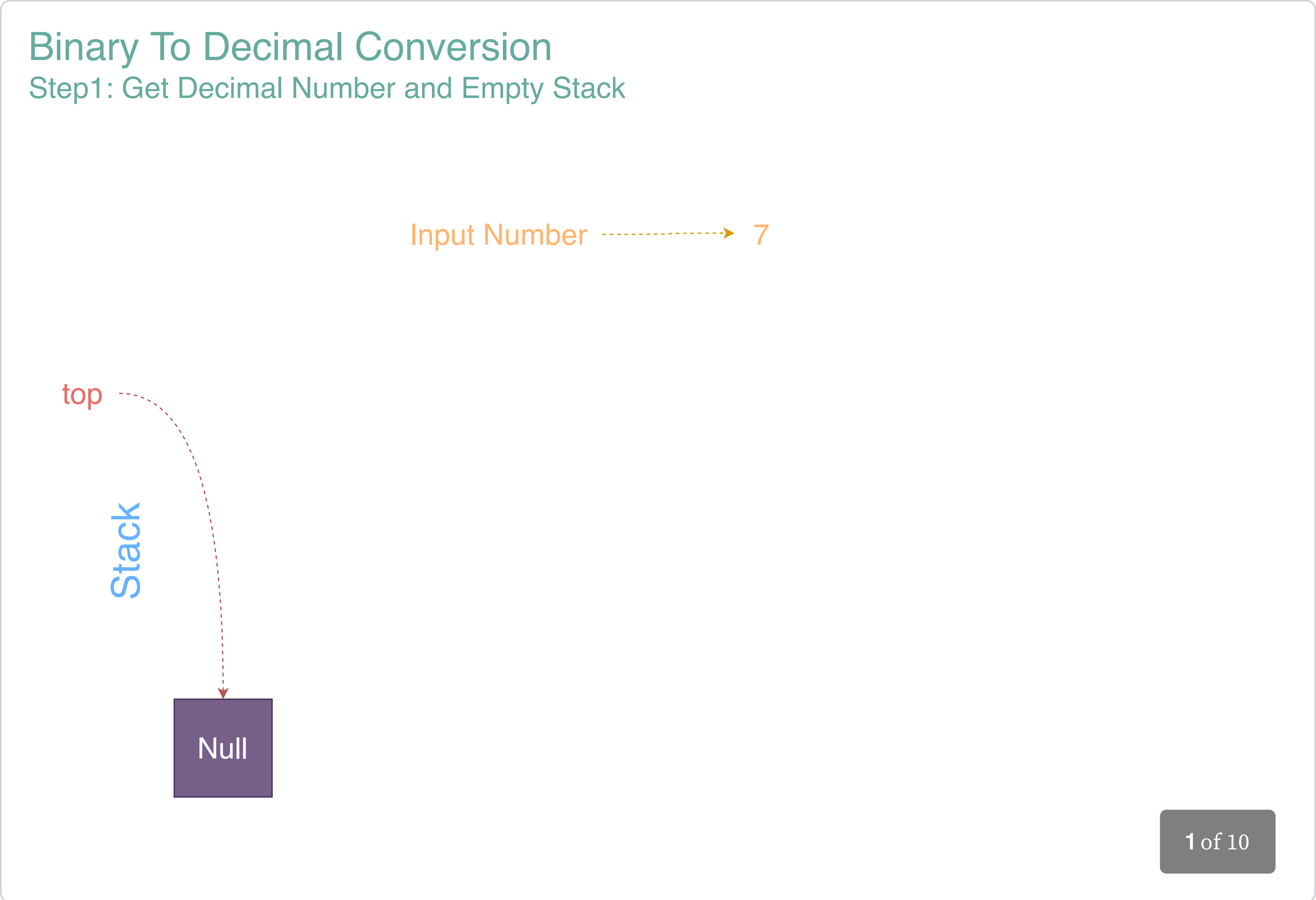
4. The number is then updated by integer division by 2 (in programming languages, this is usually denoted by `num //= 2` or `num = Math.floor(num / 2)` or `num /= 2`). This step essentially "shifts" the number one bit to the right.

5. Steps 3 and 4 are repeated until the number becomes zero.

6. At this point, the stack contains the binary representation of the number, but in reverse order. This is because the first bit we calculated (the least significant bit, or the "rightmost" bit) is on the top of the stack, while the last bit we calculated (the most significant bit, or the "leftmost" bit) is on the bottom of the stack.

7. So, the next step is to reverse this order. This is done by popping the stack until it's empty and appending each popped bit to the result. Since a stack follows "Last In, First Out" rule, this will correctly reverse the order of the bits.

8. Finally, the algorithm returns the result, which is the binary representation of the original number.


Binary To Decimal Conversion
Step1: Get Decimal Number and Empty Stack

## Code

Here is how we can implement this algorithm:

```java
import java.util.Stack;

public class Solution {
    // Method to convert a decimal number to binary representation
    public String decimalToBinary(int num) {
        // Create a stack to store binary digits
        Stack<Integer> stack = new Stack<>();
        // Create a StringBuilder to build the binary representation
        StringBuilder sb = new StringBuilder();

        // Convert the decimal number to binary using a stack
        while (num > 0) {
            stack.push(num % 2); // Push the remainder (binary digit) onto the stack
            num /= 2;           // Divide the number by 2 for the next digit
        }

        // Pop the binary digits from the stack and append them to the StringBuilder
        while (!stack.isEmpty()) {
            sb.append(stack.pop());
        }

```

You must run your code first

## Time and Space Complexity Analysis

The time complexity of this algorithm is $O(log(n))$ due to the division by 2 at each step.

The space complexity is also $O(log(n))$ because in each step we will be pushing an element on the stack, and there are a total of $log(n)$ steps.

← Previous
Problem 3: Decimal To Binary Conversion

Next →
Problem 4: Next Greater Element

☐ Mark as Completed